

Article

Explainable Recommender Systems Through Reinforcement Learning and Knowledge Distillation on Knowledge Graphs

Alexandra Vultureanu-Albiși * , Ionuț Murarețu  and Costin Bădică 

Department of Computer and Information Technology, Faculty of Automatics, Computers and Electronics, University of Craiova, 200440 Craiova, Romania; ionut.muraretu@edu.ucv.ro (I.M.); costin.badica@edu.ucv.ro (C.B.)

* Correspondence: alexandra.vultureanu@edu.ucv.ro

Abstract: Recommender systems have evolved significantly in recent years, using advanced techniques such as explainable artificial intelligence, reinforcement learning, and graph neural networks to enhance both efficiency and transparency. This study presents a novel framework, XR^2K^2G (X for explainability, first R for recommender systems, the second R for reinforcement learning, first K for knowledge graph, the second K stands for knowledge distillation, and G for graph-based techniques), with the goal of developing a next-generation recommender system with a focus on careers empowerment. To optimize recommendations while ensuring sustainability and transparency, the proposed method integrates reinforcement learning with graph-based representations of career trajectories. Additionally, it incorporates knowledge distillation techniques to further refine the model's performance by transferring knowledge from a larger model to a more efficient one. Our approach employs reinforcement learning algorithms, graph embeddings, and knowledge distillation to enhance recommendations by providing clear and comprehensible explanations for the recommendations. In this work, we discuss the technical foundations of the framework, deployment strategies, and its practical applicability in real-world career scenarios. The effectiveness and interpretability of our approach are demonstrated through experimental results.

Keywords: recommender systems; explainability; knowledge-graph; reinforcement learning; knowledge distillation; job domain



Academic Editors: Cecilia Zanni-Merk, Franco Giustozzi and Ali Ayadi

Received: 12 March 2025

Revised: 25 March 2025

Accepted: 27 March 2025

Published: 30 March 2025

Citation: Vultureanu-Albiși, A.; Murarețu, I.; Bădică, C. Explainable Recommender Systems Through Reinforcement Learning and Knowledge Distillation on Knowledge Graphs. *Information* **2025**, *16*, 282. <https://doi.org/10.3390/info16040282>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Due to the complexity of the workforce composition structure and the rise of the global economy, businesses need automatic solutions with greater precision to help them solve the problem of job recruitment overload. Although popular hiring platforms provide more convenient channels for both companies and job seekers, it is critical to establish more effective techniques for determining whether qualifications match job requirements. The matching task is essential, but in light of the data protection regulations [1], it is also important to offer an interpretation or explanation of recruitment decisions. With increased corporate rivalry in the modern labor market, better understanding and evaluating emerging risks when employing new employees has become critical. Such risks stem from a variety of factors, including unequal hiring processes, lack of appropriate training for human resources managers, and lack of finding the best match for the specific position. According to [2], one of the most important factors appears to be the danger of newly hired employees leaving the job too soon. However, it is critical for businesses to not only observe and understand

these explanations but also act proactively by forecasting with greater precision whether or not a new hire will leave their new job in the near future to avoid the related risk [3].

Recent advances in the interpretability of artificial intelligence (AI) models have contributed to the increasing adoption of machine learning (ML) and deep learning (DL) approaches by companies in the short and long term to improve their decision-making processes by incorporating them into their current and future strategies in a wide range of applications, including highly regulated areas such as medicine, finance, and recruitment [4–6]. Recent developments in AI and ML [7] provide new and better computational tools that can help businesses improve their prediction abilities about early job exits by new employees, thus better defending themselves against such emerging threats. These predictions can be made by applying recent and forthcoming ML algorithms and platforms to historical data on the demographic and professional profile of the new employees, information on the type and size of the company, and information on the geo-economic backdrop [8]. Furthermore, new developments in explainable AI (XAI) can be used to interpret businesses better understanding these ML predictions [9].

This paper introduces XR^2K^2G , a novel framework for job recommender systems (JRSs). XR^2 includes the key approaches connected to the system's functionality, where X describes *explainability* as an essential factor to comprehend recommendations, the first R for *recommender systems* (RSs), a central component and the second R for *reinforcement learning* (RL) for the recommendation process. K^2G refers to the model architecture, with first K representing *knowledge graph* (KG), second K for *knowledge distillation* (KD), and G for *graph-based methods* as graph attention networks (GATs).

Our approach integrates KG, KD, and RL to improve recommendations by exploiting KG's rich, structured data representation, which includes entities and their relationships, allowing for interpretable, context-aware recommendations that go beyond simple relationship patterns. KD transfers knowledge from a high-performing model (teacher) to a simpler and faster model (student). This minimizes computational overhead while preserving the majority of the teacher's performance. When used with KG, KD contributes to reducing high-level patterns and reasoning from a large-scale KG-based model into a smaller model for efficient inference. Through repeated interactions that optimize long-term rewards (such as user satisfaction), RL learns suitable recommendation rules that improve system performance and user satisfaction. In our approach, we employ KG to encode job titles, required skills, and user profiles, KD to compress a KG model for recommendations, and RL for the recommendation process, focusing on jobs that are expected to fit the user's skills.

Although this work focuses on integration, KG embeddings, GAT, RL, and XAI, it should be noted that the effectiveness and generalization of such approaches also depend heavily on the diversity of user-interaction data. As users leave more digital traces (e.g., skills, occupations, interests), systems receive more informative graphs, which in turn enhance the accuracy of knowledge-based reasoning and recommendation. Previous research [10] has shown that data-driven paradigms, more specifically large-scale graph-based ones, play a key role in making scalable and high-performing recommendation platforms feasible. Our system is designed to take advantage of direct modeling of user–skill–job–occupation relations so that the model can benefit from increased data availability without being manually supervised.

The rapid expansion of online recruitment platforms is suggestive of the broader trend in hiring toward data-driven decision-making. But our approach takes advantage of this trend by using a variety of user–job interaction data to offer not only accurate but also more explainable recommendations. Our approach effectively manages the complexity of large recruiting data while improving the transparency of job matching.

In this work, we used data from our previous research [11], which provide a rich and structured representation of real-world job postings and candidate profiles. This choice allows us to evaluate our framework on a practical and diverse set of job–skill–occupation relationships that mirror actual recruitment scenarios.

To demonstrate the novelty of our XR^2K^2G framework, we compare it to existing state-of-the-art frameworks such as GEPKSD—graph-enhanced PKSD (privileged knowledge state distillation) [12]—for educational recommendations and top-aware recommender distillation (TRD) [13] for top-aware ranking optimization. We analyze their design principles and methodologies to highlight the unique contributions of our approach. GEPKSD targets educational recommendations, personalizing them through privileged knowledge states and graph structure learning. However, it lacks a strong emphasis on explainability or application to the real world. TRD, on the other hand, focuses on fine-tuning the ranking of recommendations in general-purpose systems, using RL to increase user engagement by improving the top-ranked recommendations. Although TRD is highly adaptive to interaction with modern technology, it lacks explicit explainability tools and domain-specific enhancements such as KG. Our framework, XR^2K^2G , uses graph embeddings to express complex connections between occupations and skills, whereas GEPKSD enhances state encoding by modeling relationships between knowledge concepts. However, TRD does not explicitly use graph-based approaches. XR^2K^2G incorporates RL, KG, and KD to provide job recommendations, focusing on user-centric explainability with tools like local interpretable model-agnostic explanations (LIMEs) and Shapley additive explanations (SHAPs).

This paper is structured as follows. In Section 2, we review the relevant published works that are technically related to our proposed approach and also highlight additional articles that explore various aspects of the field, offering valuable information for our research. In Section 3, we present the formulation of the proposed framework for job recommendations, XR^2K^2G , which aims to efficiently offer recommendations with clear and concise explanations. In Section 4, we describe the materials and methods used to implement the proposed framework used in this study, including the description of the dataset and the experimental configuration of our system. The tools and libraries that we have used to evaluate our models are covered in this section. We present the interpretation of the results and discuss the implications of the study findings in Section 5. Finally, Section 6 concludes the paper.

2. Related Works

Explainability makes a system’s processes and decisions more understandable. In other words, it aims to ensure that clear, relevant, and understandable descriptions are made of how input variables, such as data or features, drive outputs, such as predictions, recommendations, or judgments. LIME and SHAP are important methods that help stakeholders (such as users, developers, and regulators) understand system behavior and develop trust in system decisions.

Using methods such as LIME and SHAP, RS can explain the rationale behind recommendations, making the user experience more visible and helping people to understand *how* and *why* of the recommendations they receive. LIME and SHAP can highlight the features that have the most influence on anticipating users’ decisions in RS. LIME and SHAP are techniques that can explain more complex models by building on concepts intrinsic to linear regression. In the context of RS, which often functions like black boxes, the incorporation of such explainability frameworks is important in causing trust and acceptance among users. In linear regression-based RS, the coefficients or importance of a particular feature (input variable) with higher value have a greater impact on the recommendation score [14]. In nonlinear models, these methods provide insight into the importance of features by

approximating the decision-making process of the model [15,16]. LIME and SHAP can provide context for how these features affect certain predictions [17,18]. Systems can be adjusted to meet the demands of certain users by understanding the features that influence specific recommendations.

The integration of KG, KD and RL may bring several benefits to RS, such as improving the understanding of context, providing deeper insight into user intent, enhancing interpretability and transparency, and improving personalization, by better capturing the semantics of data entities and their associated relationships, as well as the dynamics of user–item interaction.

The knowledge-aware path recurrent network (KPRN) model to improve recommendations using KGs was proposed in [19]. The KPRN framework captures the semantics of both entities and relationships to build path representations of user–item interactions. Through the use of sequential dependencies within a path, the model provides efficient reasoning on paths to support the reasoning behind a user–item interaction. Additionally, the KPRN model leverages a novel weighted pooling procedure to distinguish between the relative merits of several routes to establish a user–item connection, thus enhancing the model with a certain degree of explainability.

By providing actual paths in a KG, a novel policy-guided path reasoning (PGPR) approach combining interpretability and recommendation was proposed in [20], bringing certain contributions. First, PGPR highlights how crucial it is that recommendations are issued using KGs. Second, a unique soft reward technique, user-conditional action pruning, and a multi-hop scoring function were proposed in the context of a novel RL method. Third, PGPR proposes a new policy-guided graph search approach whose goal is to quickly and efficiently sample reasoning paths for recommendations.

In [21], an original model was developed to merge KG data into a sequential recommendation framework. This model is known as knowledge-guided reinforcement learning (KERL). The KERL model improves sequential recommendation by improving state representations with KG relationships, creating a reward function that combines sequence- and knowledge-level feedback, and implementing a novel RL algorithm that effectively explores and exploits user preferences over time.

The novel framework distilled embedding-based recommender with reinforced distillation (DE-RRD) for applying KD to RSs was proposed in [22]. The approach allows the student model to learn from both the teacher’s predictions and the latent knowledge contained in the teacher model. DE-RRD is specifically composed of two techniques: (i) distillation experts (DEs), who use the teacher model to extract latent information and transfer it directly. DE makes use of “experts” and a modern expert selection technique to efficiently reduce the extensive teacher knowledge for the student with minimal ability. (ii) The knowledge revealed by the teacher’s prediction is transferred via the relaxed ranking distillation (RRD), considering the items’ ranking order.

Our approach uses KG and RL to improve recommendations by integrating significant concepts from related works. By employing path representations for semantic reasoning and policy-guided exploration to enhance interpretability, it expands upon KPRN and PGPR. Moreover, it is inspired by KERL by incorporating KG-enhanced state representations and a reward function for RL. Lastly, our framework incorporates KD concepts borrowed from DE-RRD, thus improving the accuracy and efficiency of the model while preserving the explainability of job recommendations.

3. XR^2K^2G Framework

This section includes the main definitions and the problem formulation. Figures 1 and 2, which highlight the integration of KG, GAT, KD, and RL components, show the entire

system workflow of the XR^2K^2G framework and provide an overview of the proposed model architecture.

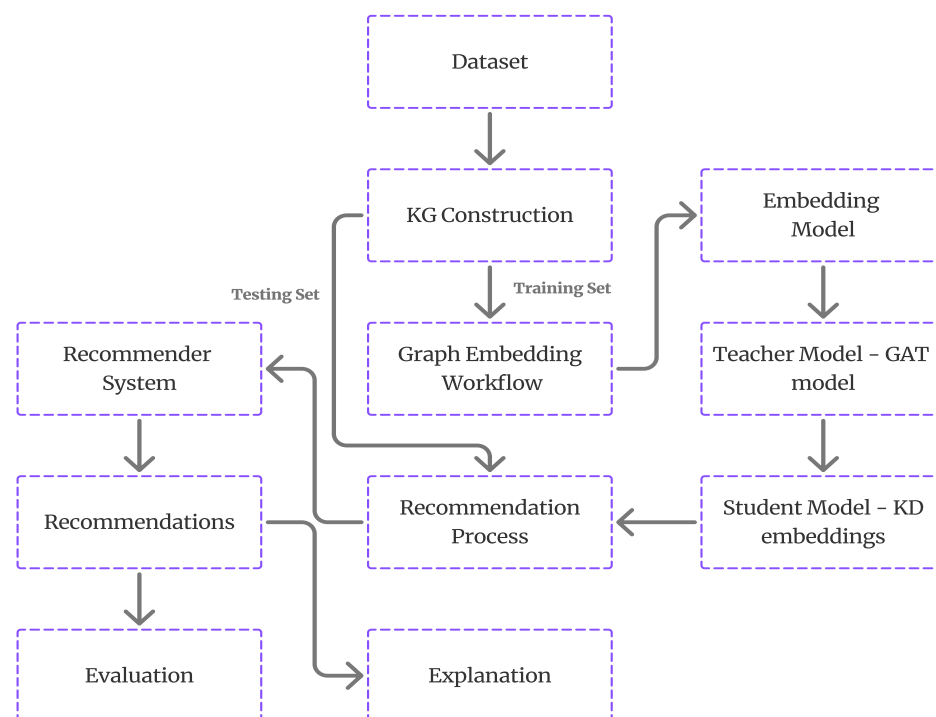


Figure 1. System workflow.

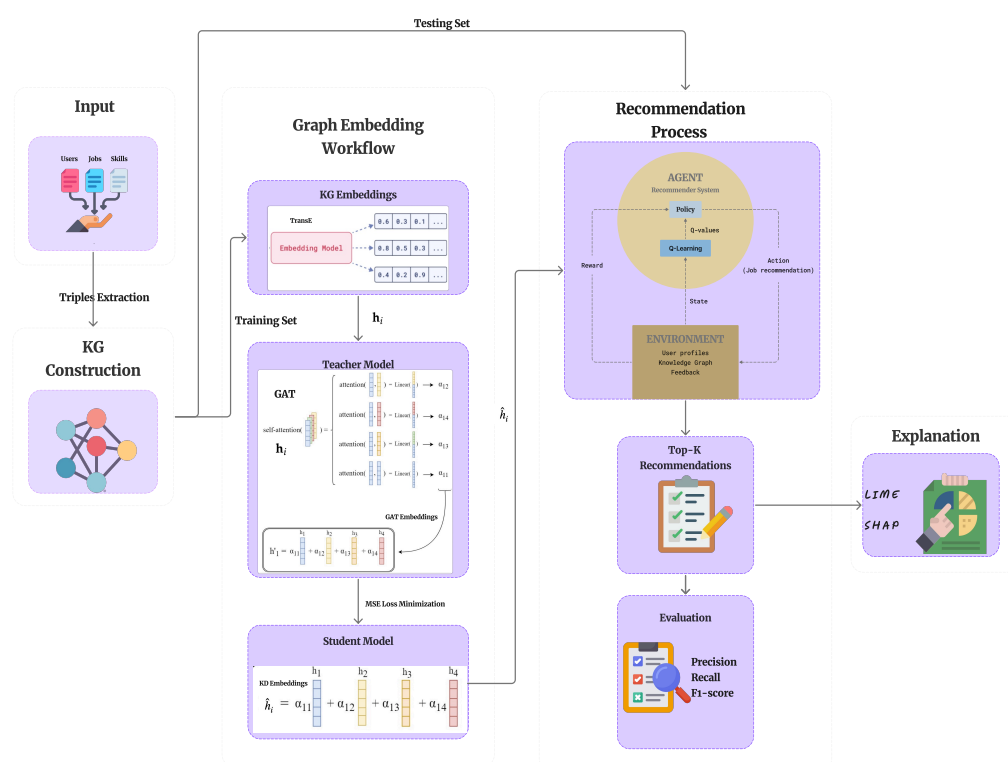


Figure 2. Overview of the proposed method XR^2K^2G .

In addition to the exploitation of a multi-relational KG, our XR^2K^2G framework introduces a new explainable solution compared to the traditional RS model based on straightforward matching between user profile and task requirement. KG encodes intensive semantic relationships between users, tasks, skills, and occupations in the form of *has_skill*,

requires_skill, and *requires_occupation*. TransE is used to represent these relations and entities, and GAT is used to strengthen them. KD is used to learn the adaptation of the user representation. The user job reward signals are then represented in dynamic recommendation policies using Q-learning. The addition of methods such as LIME and SHAP allows the system to produce understanding explanations as well as accurate recommendations.

3.1. Problem Formulation

Consider a user *user_1*, who adds their CV to the system. The CV states that *user_1* has skills. These skills are extracted and assigned to entities in the KG, which also possess relations such as “*has_skill*” and “*requires_skill*”. The history of *user_1*’s previous jobs is also mapped to entities in the KG, completing the user profile with structured data. The KG is composed of various entities that are users, jobs, skills, and occupations and the relationships among them. Both entities and relationships are embedded into high-dimensional numerical vectors (embeddings). The embeddings are also refined by a GAT model, which focuses on the most effective neighbors in the KG. This step generates context-aware embeddings for users and jobs. GAT makes embeddings context-aware by providing attention scores to neighboring nodes, prioritizing important connections, and aggregating relevant information from the KG, ensuring that each embedding captures both structural and relational significance for more precise job recommendations.

The GAT model acts as the teacher model because it can generate context-aware embeddings and uses the attention mechanism. The student model targets are the embeddings provided by the teacher model (from GAT). The student model is trained with the MSE loss function to minimize the difference between its output and that of the teacher. As a result, the student model is used as the teacher model to solve other problems such as RL recommendation generation. RL agent uses the distilled embeddings to compute the Q-values for job recommendations from the *user_1* profile. The KD approach keeps high-quality embeddings and greatly reduces the amount of computation needed to use a GAT in inference. It also helps to ensure that the architecture is suitable for scaling up applications with a large number of users and jobs.

The agent updates its policy to propose jobs with higher expected rewards, optimizing the strategy of proposing jobs over time. It begins with the agent encoding the state of the user, together with their distilled embeddings generated by the KD module. The agent then selects a job recommendation as an action given this state, using an epsilon-greedy policy: it either explores by selecting a random job or exploits its experience to recommend the job with the highest Q-value, as approximated by the Q-network. The environment provides feedback as a reward for the recommendation, highlighting how well the recommended jobs fit the user’s skill set. The agent gradually adjusts its policy and changes its Q-values in accordance with that reward by using the Q-learning update rule. The agent focuses on recommending jobs that improve user satisfaction, and trial encounters are used to modify the policy. Due to this real-time optimization, the system will learn to adapt to user needs and provide recommendations that are increasingly relevant. Once the optimal policy has been learned, the system recommends the top *K* jobs to *user_1*. To enhance explainability, the system generates explanations for recommendations using LIME and SHAP.

3.2. KG Construction

A KG is a systematic organization of facts in which the entities are related to each other, including persons, places, or objects. A KG can be mathematically defined as a directed labeled graph [23]:

$$G = (E, R, S),$$

where:

- E is a finite set of **nodes** (entities).
- R is a finite set of **edge types** (relationships).
- S is a finite set of **triples**, where:

$$S = \{(h, r, t) \mid h, t \in E, r \in R\}$$

with h the head entity (source of the edge), r the relationship (edge type), and t the tail entity (target of the edge).

The initial step in the XR^2K^2G framework is building the KG, which acts as the starting point for later phases such as embedding generation and job recommendation.

The KG is constructed using three datasets that provide the input data:

- User data (CVs) contain user profiles, including their skills and occupations.
- Skill data represent skills with unique identifiers and descriptive labels (e.g., EBSCO URLs).
- Job data include job descriptions, specifying the required skills and occupations.

Entities and relations are extracted from the input data:

- Entities (Nodes): users, skills, jobs, and occupations.
- Relations (Edges): connections between entities based on the data, such as “*has_skill*” which links users or jobs to skills, “*has_occupation*” which connects users or jobs to occupations, “*requires_skill*” which relates jobs to required skills, and “*requires_occupation*” which associates jobs to associated occupations.

The KG triples (h, r, t) are represented as $(user_1, has_skill, skill_1)$, $(job_1, requires_skill, skill_A)$, and $(job1, requires_occupation, occupation_1)$, capturing the relationships between users, skills, jobs, and occupations. In Figure 3, the KG structure is illustrated, with nodes representing users, skills, jobs, and occupations, as well as the relationships between them, such as “*has_skill*”, “*requires_skill*”, and “*requires_occupation*”, represented as directed edges.

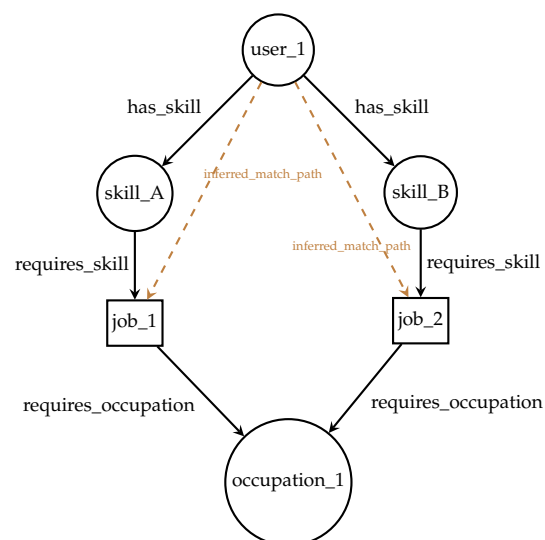


Figure 3. Example of KG representation with inferred match paths.

After the KG is constructed, we have 15,259 entities (nodes) and 11,302 relationships (edges) that summarize interconnected information from users, skills, jobs, and occupations.

KG and RS can be viewed with a mathematically focused approach that focuses on the structures and algorithms used to make personalized recommendations. RS combined with KG enhances the recommendation process using hierarchical relationships encoded in KG. Integration with KG improves user and item representations with context information

and provides more personalized recommendations than explicit user–item interactions. In addition, diversity is embedded within the recommendations of KG following less obvious links, and explainability was significantly enhanced since more transparent paths can facilitate the substantiation of recommendations.

3.3. Graph Embedding Workflow

KG embeddings are low-dimensional vector representations of KG’s entities and relationships [24]. These embeddings represent both structural and semantic information in continuous vector spaces.

A common KG embedding process usually consists of three steps:

1. Entities and relations

Each entity ($e \in E$) and the relationship ($r \in R$) in the KG is represented as a vector in a continuous d -dimensional space ($\mathbb{R}^d$):

$$e \mapsto \mathbf{e} \in \mathbb{R}^d, \quad r \mapsto \mathbf{r} \in \mathbb{R}^d.$$

Suppose that we have a user *user_1*, a skill *Python*, and a job *Data_Scientist* in the KG, we will have relationships such as (*user_1*, *has_skill*, *Python*) and (*Data_Scientist*, *requires_skill*, *Python*).

2. Scoring function

In our framework, we use **TransE** model and the loss function is typically a margin-based ranking loss, also known as a pairwise ranking loss:

$$\mathcal{L}_{\text{TransE}} = \sum_{(h,r,t) \in S} \sum_{(h',r,t') \in S'} \max(0, \gamma + f(h,r,t) - f(h',r,t')) \quad (1)$$

where S is the set of positive triples (KG relationship), S' is the set of negative samples generated by replacing either h or t , γ is the margin hyperparameter, ensuring that positive triples have lower distances than negative ones, and **TransE** scoring function is defined as follows:

$$f(h,r,t) = -\|\mathbf{h} + \mathbf{r} - \mathbf{t}\|,$$

where $\|\cdot\|$ is the vector norm (e.g., L_1 or L_2 norm).

TransE is a straightforward, efficient, and interpretable technique that struggles with one-to-many relationships. TransE is a fundamental KG embedding model that has resulted in numerous modifications, including TransH, TransR, and TransD, that address its shortcomings while maintaining the primary translation process [25].

We apply this function to the triples extracted from KG (*user_1*, *has_skill*, *Python*). If the calculated score is high, it indicates that this triple is valid and that the model has learned a strong relationship between the user and the skill, which means *user_1* probably has the skill *Python*.

3. Learning entity and relation representations involve defining how they are expressed as continuous vectors.

Example of a positive triple: (*user_1*, *has_skill*, *Python*) has a high score, and an example of a negative triple: (*user_1*, *has_skill*, *Snorkeling*) has a low score. Over time, the embeddings are adjusted so that the valid relationships in the KG are properly represented in the vector space.

The GAT module accepts as input the information of the KG from the generated embeddings. GATs are an extension of graph neural networks (GNNs), which explicitly incorporate attention processes [26,27]. This allows the model to assign attention weights to neighbors while aggregating their attributes. GAT generates embeddings that combine

node relations and the structure of the KG, enabling improved contextual and more meaningful entity (users, jobs, skills) representations in vectors. The attention scores enable GAT to capture those relations or neighboring entities with greater relevance to a particular entity. For example, it would illustrate why a specific skill is important when recommending jobs.

Let $\mathbf{h}_i \in \mathbb{R}^d$ represent the feature vector of node i , where d represents the dimensionality of the feature vector associated with node i , and let $\mathcal{N}(i)$ denote the input set of neighbors of node i . The value of d determines how much information can be encoded about each node. The larger d allows for capturing more complex patterns but also increases the computational cost. In our case, d is often chosen during the initialization of the graph network.

The goal of GAT is to compute a new representation $\mathbf{h}'_i$ for node i by aggregating features from its neighbors.

For each pair of connected nodes (i, j) , an attention score e_{ij} is calculated to measure the importance of the features of node j to node i . This is given by:

$$e_{ij} = \sigma(\mathbf{a}^\top \cdot [\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j]),$$

where σ is chosen as the activation function *Leaky Rectified Linear Unit (LeakyReLU)* in our framework, $\mathbf{W} \in \mathbb{R}^{d' \times d}$ is a trainable weight matrix (transforms input features to output features), $\mathbf{a} \in \mathbb{R}^{2d'}$ is a learnable attention vector (determines the importance of neighboring nodes dynamically during the attention computation process), and $\parallel$ denotes the concatenation of the vector.

Attention scores are normalized using the softmax function to ensure that they sum to 1 over all neighbors of node i :

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})}.$$

The final representation of node i is computed as a weighted sum of its neighbors' features, scaled by the attention coefficients:

$$\mathbf{h}'_i = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \cdot \mathbf{W}\mathbf{h}_j \right),$$

where σ is a non-linear activation function (as *exponential linear unit (ELU)*).

For better stability and representation learning, GAT leverages multi-head attention. Using K attention heads, the node representation is computed as:

- **Concatenation:**

$$\mathbf{h}'_i = \parallel_{k=1}^K \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k)} \cdot \mathbf{W}^{(k)} \mathbf{h}_j \right),$$

where $\parallel$ denotes concatenation.

- **Averaging:**

$$\mathbf{h}'_i = \frac{1}{K} \sum_{k=1}^K \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k)} \cdot \mathbf{W}^{(k)} \mathbf{h}_j \right).$$

The result is a refined embedding $\mathbf{h}'_i$ for each node, which captures both the node's features and the importance-weighted aggregation of its neighbors' features. The input for the KD module is the refined embeddings ($\mathbf{h}'_i$) produced by GAT.

The goal of KD is to use the knowledge from a larger model called *teacher* to train a smaller model called *student* [28]. The basic idea is to have the student model copy the teacher model; in this case, the features, activations, logits, and neurons may all be thought of as knowledge that guides the student model's learning. The primary goal is to minimize a loss function that includes both the typical cross-entropy loss with ground-truth labels and a distillation loss that evaluates the difference between student and teacher predictions.

The KD module takes as input the refined embeddings ($\mathbf{h}'_i$) generated by the GAT module. The teacher model in the KD module is the GAT model trained on the KG data. The student model learns to approximate the refined embeddings ($\mathbf{h}'_i$) generated by the teacher model while starting from the TransE embeddings ($\mathbf{h}_i$). The student model produces distilled embeddings, denoted as $\hat{\mathbf{h}}_i$.

The student model learns a mapping function $f_\theta(h'_i)$:

$$\hat{\mathbf{h}}_i = f_\theta(h'_i)$$

where:

- $\hat{\mathbf{h}}_i$ is the distilled embedding;
- $f_\theta(h'_i)$ is the mapping function (a neural network) trained to approximate.

The training process minimizes a mean squared error (MSE) loss:

$$\mathcal{L}_{\text{KD}} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{h}'_i - \hat{\mathbf{h}}_i\|^2,$$

where $\mathcal{L}_{\text{KD}}$ represents the KD loss, N the total number of entities (e.g., users, jobs, skills, occupations) in the dataset, $\mathbf{h}'_i$ the refined embedding of entity i generated by the teacher model (e.g., GAT) and $\hat{\mathbf{h}}_i$ the embedding of entity i generated by the student model.

The distilled embeddings ($\hat{\mathbf{h}}_i$) produced by the KD module are suitable for the next step in RL-based job recommendation.

3.4. Recommendation Process

The recommendation process in our framework is modeled as an RL problem, where the RS acts as an agent that interacts with the environment to optimize the job recommendations. The following is a detailed description of the process, highlighting all components of the RL.

Environment

The environment represents the world in which the agent acts. The environment responds to the agent's activities by presenting him with new situations. It also delivers signals that tell the agent how well it is doing in terms of meeting its objectives.

The environment in XR^2K^2G consists of the static user base, KG, and feedback mechanisms. The static user base includes user profiles such as skills, occupations, and distilled embeddings ($\hat{\mathbf{h}}_i$), providing personalized information specific to each user. Feedback mechanisms determine rewards based on how well the recommendations match the user's preferences. The user interacts with the recommended jobs and gives feedback using a simulated environment to evaluate the relevance of the recommendations through rewards based on predefined logic. The environment updates the state (user profile) and provides rewards based on such interactions.

Agent

Agent represents the decision-maker whose primary goal is to determine the appropriate policy for action selection that maximizes cumulative reward over time.

In XR^2K^2G , RS is the agent. First, the agent does not know the user's preferences and checks job recommendations in a random way. Its goal is to choose job recommendations (actions) that maximize user satisfaction, represented by cumulative rewards. The cumulative reward (r) for every recommendation action taken by the RL agent is calculated with immediate rewards. The immediate reward depends on how well the user's skills match the required skills of the recommended job:

- (i) $r = 1$ in case of the overlap between the user's skills and the job required skills;
- (ii) $r = 0$ in case of non-overlap.

The cumulative reward (G_t) is defined as the discounted sum of immediate rewards over time:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1},$$

where γ is the discount factor ($0 \leq \gamma < 1$), which determines the relative importance of future rewards and r_{t+k+1} the immediate reward at step $t + k + 1$.

The agent learns its policy by learning a simulated reward and not by modeling user activity. The agent uses learning a stochastic policy by using the epsilon-greedy policy to select actions:

- (i) Exploration: With probability ϵ , the agent selects jobs randomly to explore new possibilities.
- (ii) Exploitation: With probability $1 - \epsilon$, the agent selects the job with the highest Q-value based on its current policy.

The reward is provided based on the similarity between the user's skills and the skills required for the job. In addition, the relevance of the occupation is considered: If the recommended job's required occupation aligns with the user's occupation, an additional reward is given. When a job closely matches the user profile, the system assumes that the recommendation was successful. The user does not actively "take actions" in response to recommendations, such as selecting items or updating their profile. Instead, the system operates on the basis of pre-existing user profiles (skills and occupations). Since the user does not actively take actions, the system relies on cosine similarity between the user's embedding and the job's embedding:

$$\text{cosine_similarity}(u, j) = \frac{\hat{\mathbf{h}}_u \cdot \mathbf{h}_j}{\|\hat{\mathbf{h}}_u\| \|\mathbf{h}_j\|}, \quad (2)$$

where u represents the user, $\hat{\mathbf{h}}_u$ the distilled embedding of the user, j represents the job, and $\mathbf{h}_j$ is the embedding of the job.

The cosine similarity score serves as a proxy reward, where a higher similarity indicates a better match and results in a higher reward.

Policy

The policy $\pi(a|s)$ represents the probability π of taking action (a) given that the agent is in state (s). Based on policy, the RL algorithms are policy or non-policy [29]. On-policy is where the agent learns directly from the actions it does in accordance with its present policy. Off-policy focuses on a separation of behavior and target policies, similar to Q-Learning, in which the agent learns the optimal Q-values independent of the actions it takes. Q-learning is a model-free RL method and allows learning of the optimal action selection policy for a given finite MDP [30]. The idea is to learn the expected value of a certain action, in any given state, through the use of the Q-value function and then to perform the optimal behavior.

The framework adopts an off-policy approach, where the learned policy is optimized independently of exploratory actions, as in Q-Learning.

XR^2K^2G uses the Q-function to calculate the long-term expected benefit of recommending jobs to users based on their profile. Consider both immediate reward for the action and future rewards, calculating the overall worth of selecting action a (job recommendation) in state s , where $s = [\hat{\mathbf{h}}_i, \text{previous_jobs}]$, $\hat{\mathbf{h}}_i$ is the distilled embedding and previous_jobs the set of jobs previously recommended.

The goal is to find the optimal Q values $Q(s, a)$, and the Q learning algorithm iteratively updates the Q values using the following update equation.

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (3)$$

where α is the learning rate that controls how much new information overrides old information, r is the reward received after taking action a in state s , and s' is the state that results from taking action a .

For a given user, the Q-function computes the Q-value for all possible job recommendations, and the Q-values $Q(s, a)$ for all jobs are ranked in descending order. Greater Q-values indicate jobs with greater expected long-term rewards based on the user's profile and interests. To show the most valuable jobs to the user, XR^2K^2G uses the top K jobs where the highest Q-values K are selected as recommendations.

3.5. Explanation Generation

Following the top K recommendation step, an explanation is generated to provide users with the reasoning behind the recommendations. The system produces an explanation that is understandable to humans on the basis of the output of SHAP and LIME. The framework improves user satisfaction and trust by ensuring that explanations are interpretable with the use of LIME and SHAP. The framework retrieves the relevant features for the user and the recommended job.

LIME perturbs the input state (ss) and observes changes in Q-values to identify the most influential features that contribute to job recommendations. The LIME approach provides an explanation for a specific instance, where an instance refers to the input data being analyzed. The process begins by making slight modifications to the instance's features to create synthetic samples that simulate variations in the initial state. The model then evaluates these perturbed samples and generates predictions for each. A local interpretable model is trained on the perturbed data to approximate the behavior of the complex Q-learning model near the instance. Finally, LIME generates an explanation by highlighting the most important features that influenced the job recommendation, making the decision process more interpretable.

SHAP calculates each feature's contribution to the Q-value and provides a ranked list of features with both positive and negative contributions. The process begins with the prediction of the model for a specific instance. Various subsets of attributes are examined to see how they individually and collectively contribute to the model's prediction. These combinations are then used to calculate the impact of adding or removing each feature. The average contribution of each attribute is calculated across all possible combinations, ensuring that importance is assigned fairly and consistently. Finally, an explanation is created to help us understand the model's prediction.

3.6. Evaluation

The performance of the framework was evaluated using evaluation metrics, such as precision, recall, and F1-score.

- *Precision* measures the proportion of recommended items that are relevant to the user.

$$Precision = \frac{|\text{Relevant Items} \cap \text{Recommended Items}|}{|\text{Recommended Items}|}$$

- *Recall* measures the proportion of all relevant items that are recommended to the user.

$$Recall = \frac{|\text{Relevant Items} \cap \text{Recommended Items}|}{|\text{Relevant Items}|}$$

- The *F1-score* is the harmonic mean of *Precision* and *Recall*, providing a balance between the two.

$$F1\text{-score} = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

- Normalized discounted cumulative gain (*NDCG*) evaluates the quality of the ranking of the recommended items, considering the positions of relevant items in the recommendation list.

$$NDCG_k = \sum_{i=1}^k \frac{2^{\text{rel}_i} - 1}{\log_2(i + 1)}$$

4. Experimental Setup

4.1. Dataset

Two key sources—described in paper [11]—are the source of our dataset. The first source is a comprehensive classification system that provides an organized framework by mapping the connections between skills and occupations. This approach connects all occupations to both mandatory and voluntary skills, and the other way around. The second source consists of user-related data, including open online course descriptions, job vacancy descriptions, and candidate resumes. Together, these sources provide a structured and diverse dataset [31].

The dataset used in this study comprises three primary components stored in JSON files: CVs, skills, and job vacancies. CV data include details about individuals, specifically their names, skills, and occupations. Each entry has a title, the skills required for the role, and, if necessary, the occupations that must be required. The entire dataset is being converted into entities, relations, and triples, which represent users, skills, and job vacancies as unique individual items and relate them through the entity in terms of “*has_skill*”, “*has_occupation*”, “*requires_skill*”, and “*requires_occupation*”. A processed triple might contain a relationship like (user_123, “*has_skill*”, skill_1), which means that user_123 has that skill_1.

Our pre-processing pipeline parses structured data from real resumes and job postings to create a multi-relational KG. The relations *has_skill* and *has_occupation* are used to link each user to appropriate skills and occupations, while the relations *requires_skill* and *requires_occupation* are used in job postings to relate to necessary skills. Both the job seeker and the employer sides are represented by means of direct modeling of the job-side constraints. This improves the accuracy and reliability of the recommendations’ training by eliminating unlinked or undefined skills and keeping only valid entity links, which also allows us to identify and manage sparsity or noise during the construction of the KG. The foundation of embedding generation, these pre-processing processes enable precise and comprehensible downstream reasoning in our XR^2K^2G architecture.

The distribution of the number of skills per user is centered around 17–19 skills, as shown in Figure 4, with a long tail extending to individuals with fewer or many more skills.

Most users have a reasonable amount of skills, and some have more than 50. Skills common with most users include English, Greek, and French. There are a number of programming languages, such as Python 3.10 and C++ (see Figure 5). It denotes a high demand for language proficiency and programming skills among user communities.

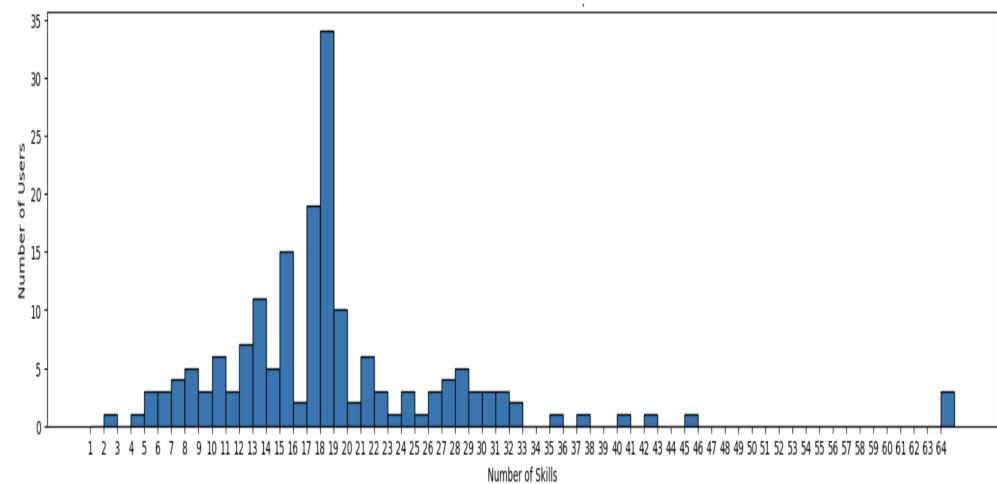


Figure 4. Distribution of number of skills per user.

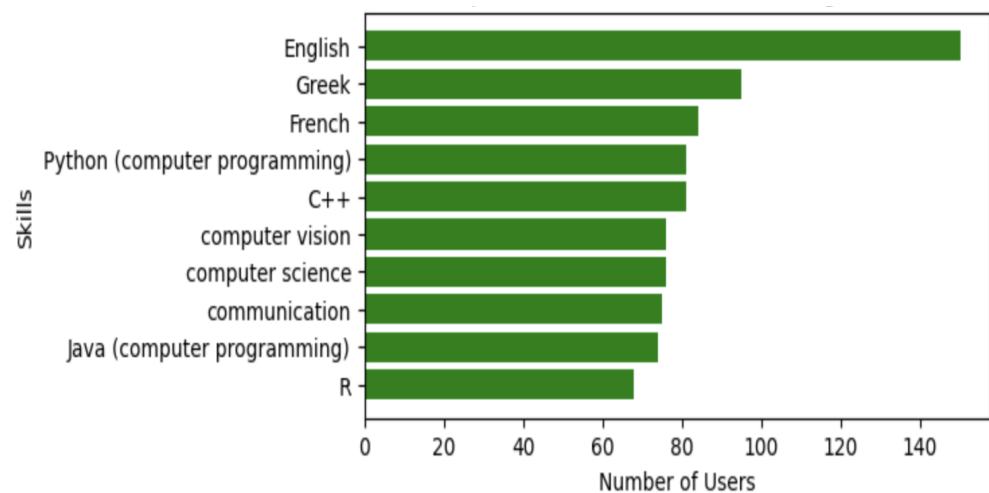


Figure 5. Top 10 most common skills among users.

Job advertisements emphasize technology and creative thinking as the most important abilities, with computer technology, creative thinking, and communication skills all in great demand (as shown in Figure 6). However, its overall most significant gaps between supply (user skills) and demand (job needs) are likely to widen in technical skills related to computer technology, communication, and computer science (as shown in Figure 7). This gap highlights the opportunities for users to improve their skills in these areas in order to meet the demands of the labor market. Jobs often require fewer skills, and many mention only one or three. However, other occupations require a broader set of skills, underlining the importance of users being well rounded in order to qualify for these more difficult tasks.

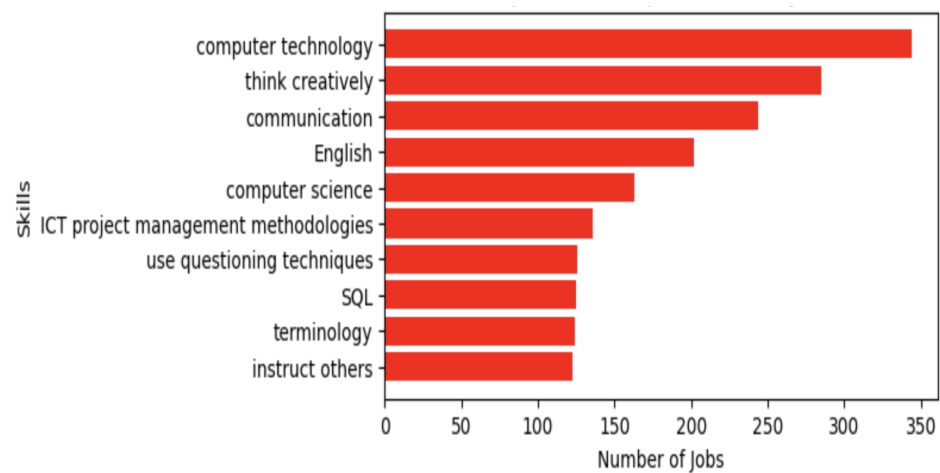


Figure 6. Top 10 most required skills in jobs.

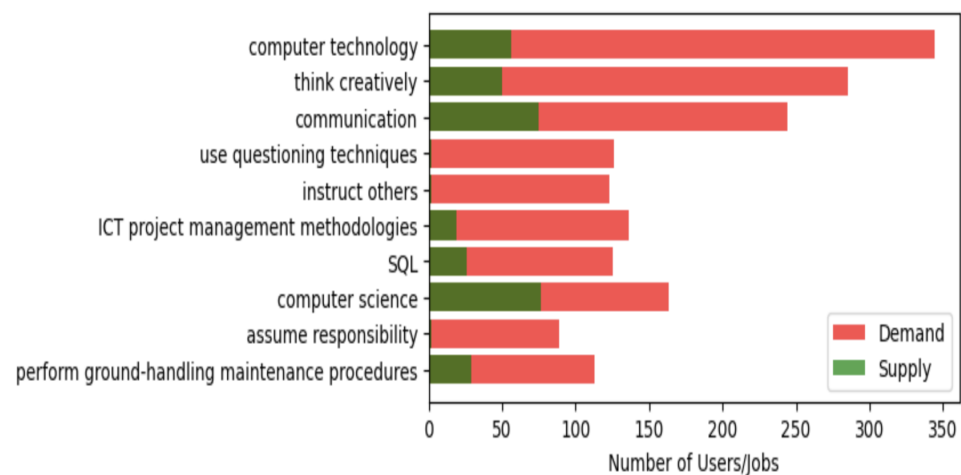


Figure 7. Top 10 skills with highest demand–supply gap.

4.2. Libraries and Tools

The experiments presented in this study were conducted using the *Jupyter Notebook* 8.26.0 application. Importing JSON files involves preprocessing the dataset. After loading the CV, skills, and job vacancies, we processed the data to extract information.

Pandas and *Numpy* were added to further manipulate the data into entities, relations, and triples. That is, skills, CVs, and job vacancies process a KG that includes users, skills, occupations, and job vacancies in entities and describes the relations connecting them.

For constructing, modifying, and analyzing complex networks with nodes and edges, we used *NetworkX*, a powerful Python library. For training the TransE model on the generated triples, we also included the PyKEEN library. The scalable, high-performance graph-based deep learning libraries PyTorch 2.3.1 and PyTorch Geometric 2.6.0. were implemented in our framework so that GNNs such as the GAT model could be used. This is suitable for complex graph-based calculations and is well integrated with all deep learning frameworks. Matplotlib 3.9.1 serves data visualization purposes.

To provide interpretable and model-agnostic local explanations, we used packages *LIME* and *SHAP*.

4.3. Hyperparameter Tuning

The following hyperparameters have been defined for tuning. To adjust the size of the embedding vectors for entities in the knowledge graph, we select values from

[50, 100, 150, 200]. For learning rates, we have multiple step sizes for gradient descent optimization that include [0.001, 0.01, 0.05]. We employ optimization methods such as Adam and SGD. Each iteration uses a different amount of samples according to the batch size [64, 128, 256]. For regularization parameters, we have L2 (weight decay), a standard form of regularization that prevents overfitting by penalizing large weights with values from $[0, 1 \times 10^{-4}, 5 \times 10^{-4}]$, and L1 (L1 coefficient), an alternative regularization technique with values from $[0, 1 \times 10^{-4}, 1 \times 10^{-3}]$.

The plot comparing the loss for different optimizers (Adam and SGD) demonstrates that Adam consistently outperforms both regularization approaches (L1 and L2) (Figure 8). Adam outperforms the SGD optimizer in terms of loss values, especially under L2 regularization.

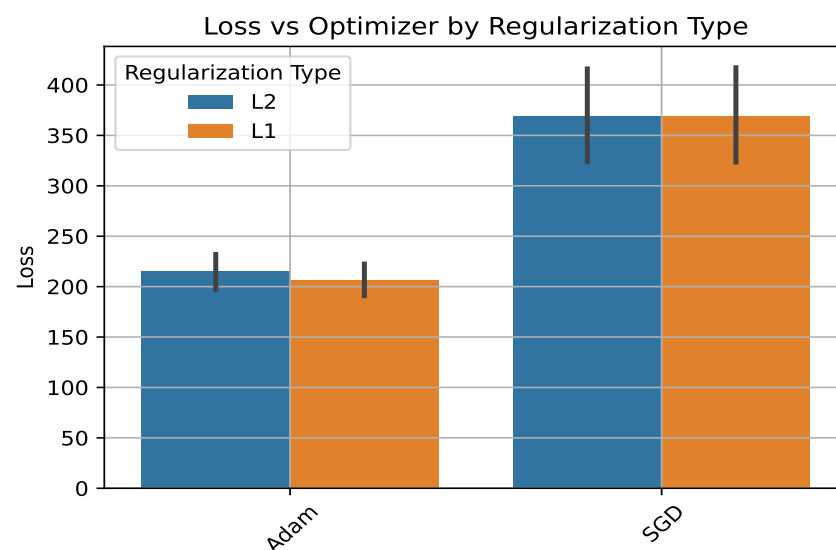


Figure 8. Loss vs. optimizer by regularization type.

As we increase the weight decay (L2 regularization), the loss increases, especially for the SGD optimizer (Figure 9). Adam adjusts well with an increase in weight decay, retaining relatively moderate loss values across different decay levels. This implies that Adam is less vulnerable to different degrees of regularization, whereas SGD is more sensitive to weight decay.

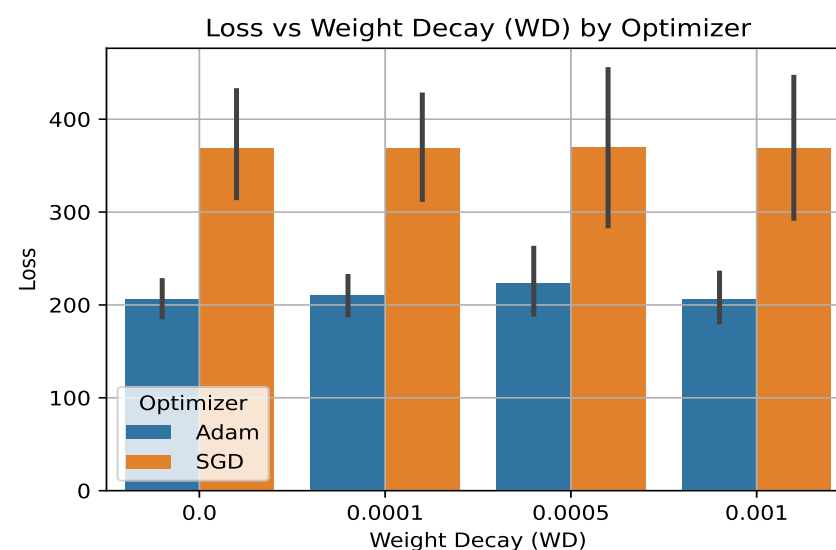


Figure 9. Loss vs. weight decay (WD) by optimizer.

In the comparison of loss and batch size (Figure 10), we can observe that increasing batch size tends to increase loss, particularly for larger values like 256. The regularization strategies (L1 and L2) exhibit this pattern, with L1 performing somewhat worse overall. This could indicate that smaller batch sizes result in more frequent updates, which leads to greater optimization results.

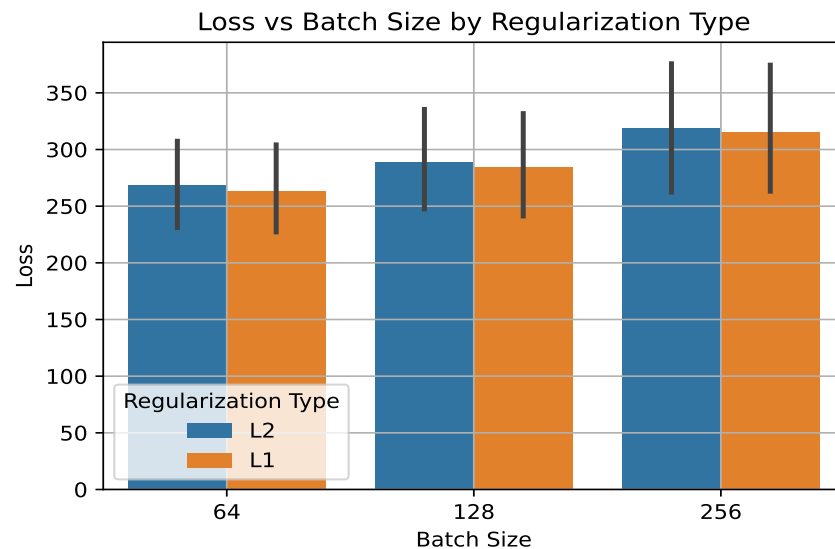


Figure 10. Loss vs. batch size by regularization type.

As the learning rate increases, we see a non-linear response in loss values (Figure 11). For low learning rates (e.g., 0.001), the loss is minimal, especially for Adam. As the learning rate increases to 0.01 and 0.05, the loss values for Adam and SGD increase, with SGD becoming especially unstable at higher learning rates. This emphasizes the importance of adjusting the learning rate for optimal performance, and lower learning rates are generally advantageous for this problem.

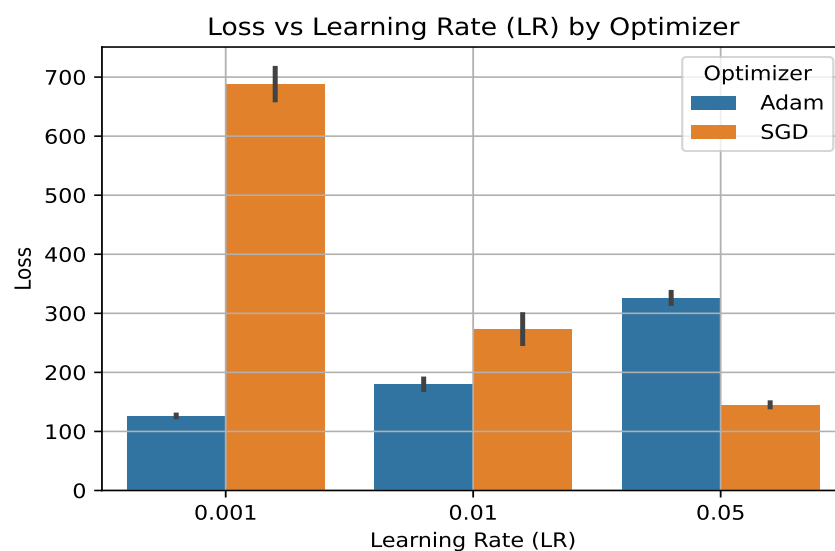


Figure 11. Loss vs. learning rate (LR) by optimizer.

Increasing the embedding dimension generally stabilizes the loss values at lower levels, particularly for Adam (Figure 12). This shows that larger embedding dimensions acquire more information, improving the optimizer in minimizing loss. However, beyond a certain point, increasing the embedding dimension may not result in significant gains in loss reduction.

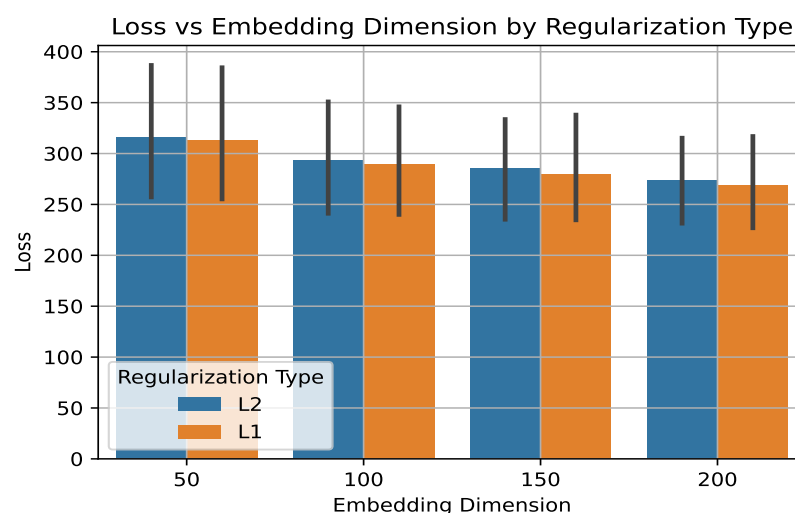


Figure 12. Loss vs. embedding dimension by regularization type.

The optimal overall configuration was obtained with L2 regularization, an embedding dimension of 50, a learning rate of 0.001, the Adam optimizer, and a batch size of 64. The ideal regularization parameter (weight decay) was 0.0001, obtaining the lowest test loss of 113.5095. This setup achieves a balance between regularization efficiency and model capacity, resulting in the greatest overall performance of the examined hyperparameter sets.

4.4. Implementation Details

Data loading and pre-processing details are as follows. The data for users (CVs), skills, and job vacancies were retrieved from *cvs.json*, *skills.json*, and *jvs.json*, respectively. Entities include users, jobs, skills, and occupations, while relations include *has_skill* for user–skill relationships, *requires_skill* for job–skill relationships, *has_occupation* for user–occupation, and *requires_occupation* for job–occupation.

KG triples are formed by combining user–skill, job–skill, user–occupation, and job–occupation links. Skills are assigned numerical indices that allow them to be used as features. PyKEEN’s *TriplesFactory* is used to manage the triples, which are divided into training (85%) and testing (15%) sets. The training set is used to learn the KG embeddings and refine them using KD, resulting in the graph embedding flow. The test set is used to evaluate these embeddings, followed by an evaluation of the job recommendation process to ensure that the Q-learning model selects the best job recommendations.

The KG is represented as an adjacency matrix (a sparse matrix).

The Q-network is used to determine the optimal actions (job recommendations) for users. The input state is the user embedding from the distilled model. Each action represents a job recommendation, and the network comprises two fully connected (FC) layers. The first FC layer has 128 hidden units activated by ReLU, whereas the second FC layer produces Q-values for all activities (job recommendations).

The default parameter settings for all experiments are as follows. For the KG embedding model-*TransE*, we set the embedding dimension at 50, the number of epochs at 100, the batch size at 64, and the learning rate at 0.0001. For the GAT model setup, there are two layers: the first layer employs four attention heads to map input features to hidden channels, and the second layer aggregates the features from the hidden channels to produce the final embeddings. For this model, we apply 64 hidden channels, an ELU for the activation function, a mean squared error (MSE) for reconstructing the embeddings, Adam with a learning rate of 0.005, and 50 training epochs.

Following GAT training, KD is used to translate the learned embeddings from GAT into a more simplified student model. The student model configuration employs two fully

connected (FC) layers, with the first FC layer having 64 hidden units and using ReLU activation, and the second FC layer producing the output. Over 100 epochs, the student model is trained to minimize the MSE loss between its outputs and the GAT-refined embeddings using the Adam optimizer.

For Q-learning parameters, we have a discount factor of 0.9 (to emphasize long-term rewards), an exploration rate of 0.1 (10% exploration, 90% exploitation), Adam as the optimizer with a learning rate of 0.001, and 1000 episodes to train the Q-learning agent. Q-values are used for recommending jobs based on user embeddings. The top K ($K = 5$) occupations are recommended based on the highest Q-values.

5. Discussion

5.1. Performance Results

To improve the transparency of the model, we chose to exemplify the prediction, for example, and thus we will present the recommendations with the explanations provided.

Our recommendation algorithm generated job recommendations based on the user profile *user_User_99*. The evaluation metrics show a precision of 0.80, indicating that 80% of the recommended occupations are consistent with the skills and interests of the user. However, the recall is 0.01, indicating that the algorithm only found 1% of all relevant jobs for the user. The F1-score of 0.02, which balances precision and recall, suggests an opportunity for improvement in the system's ability to identify relevant opportunities. The NDCG is strong at 1.00, showing that recommended jobs are relevant and well ranked based on user preferences.

The low recall means that it does not capture all relevant possibilities for users. The model only recommends a small number of jobs (e.g., top 5), which limits recall because it is impossible to recommend all relevant occupations in such a small list. Even if all the recommended jobs are relevant, the recall will be low because many other relevant jobs are not recommended. The model may not capture the full range of the user's interests or skills, resulting in a limited number of recommendations.

5.2. Explanations Results

Table 1 shows the top five job recommendations, together with their associated Q-values and important skills found using LIME explanations. The Q-value represents the degree of importance or priority of each recommendation, with higher values indicating larger possibilities. We can understand the recommendations based on the SHAP values provided for each job, as we can see in Table 2. SHAP values quantify the effect of each skill on the model's forecast for that particular recommendation. Positive SHAP values indicate that a skill improves the chance of the recommendation, while negative SHAP values indicate that a skill decreases it.

We present how we can interpret the recommendations by providing the following analysis.

- For recommendation 1 with LIME explanation, *job_System_Engineer_-_Athens* has the highest Q-value at 8.78. Positive factors include "lead a team" and "Basque", indicating leadership, and Basque language skills enhance fit for this role. Negative factors such as "setting prices of menu items", "design user interface", and "data mining" suggest that these skills are less relevant for this role. SHAP results show positive contributions from skills in alter management (+0.51), literature (+0.35), and knowledge of financial products (+0.31), indicating that adaptability, communication, and financial knowledge may be critical to the role of system engineering with financial systems. Negative contributions include soldering techniques (−0.38) and managing supplies (−0.34), suggesting that these hands-on skills are less aligned with the job.

- For recommendation 2 with LIME explanation, *job_DevOps_Architect_[50,000_-70,000_GBP]*, Slough has a Q-value of 1.63. Positive contributions include “program work according to incoming orders” and “customs law”, while “circular economy” and “direct inward dialing” may be less relevant to the user’s skills. SHAP results indicate that financial product knowledge (+0.010) and customer communication (+0.08) contribute positively, indicating a need for client engagement skills. Negative contributions, such as questioning techniques (−0.09), literature (−0.08), and Haskell (−0.08), suggest that skills such as Haskell programming may improve suitability.
- For recommendation 3 with LIME explanation, *job_Senior_.NET_Software_Engineer* has a Q-value of 1.42. Positive contributions include technical skills such as “energy efficiency”, “customer service”, “iOS”, and “production processes”. SHAP explanation highlights data gathering (+0.12), typography (+0.10), and customer communication (+0.09) as beneficial. Negative contributions include questioning techniques (−0.13) and lack of knowledge of Apache Tomcat (−0.09), suggesting that improving server technology skills may enhance fit.
- For recommendation 4 with LIME explanation, *job_Care_Assistant, Tonbridge* has a Q-value of 1.27. Positive contributions include “represent the organization”, “surveying”, and “strategic planning”. Negative contributions like “perform ground-handling maintenance procedures” imply that these skills may not be needed for the role. SHAP results indicate positive contributions from data collection (+0.13), familiarity with Absorb (learning management systems) (+0.010), digital printing (+0.08), and customer insight (+0.06), which align well with the needs of a care role.
- For recommendation 5 with LIME explanation, *Job_Localities_Social_Worker_-_Low_Case-load, Berkshire* has a Q-value of 1.25. Positive contributions include “robotics” and “file documents”, highlighting useful technical and administrative skills. Negative factors such as “physics”, “logistics”, and “Slovak” suggest some misalignment with job expectations. SHAP results point to positive contributions from digital printing (+0.08), familiarity with Absorb (+0.07), and customer satisfaction (+0.06), while questioning techniques (−0.09) and OCR (−0.06) are less emphasized.

In general, the recommendations vary in compatibility with the user profile. High Q-values, as in recommendation 1, indicate strong matches. Positive contributions emphasize user strengths relevant to job requirements, while negative factors highlight gaps or less relevant areas. Developing skills that consistently contribute positively, such as leadership, technical expertise, and organizational capabilities, can improve compatibility with job opportunities.

To provide a clear understanding of the explanation of job recommendation, we present a detailed explanation of job recommendation 1 shown in Figure 13. This explanation includes insights from both SHAP and LIME analyses, highlighting the most influential skills that contributed positively and negatively to the recommendation. The information is presented in the following box to enhance readability and emphasize key points.

Table 1. Recommendations and LIME explanations.

Recommendations	Q-Value	Positive Contributions	Negative Contributions
<i>job_System_Engineer_</i> <i>_Athens</i>	8.78	- lead a team (+0.03) - Basque (+0.02)	- set prices of menu items (−0.03) - design user interface (−0.03) - data mining (−0.03) - astrology (−0.02) - represent the company (−0.02) - mechatronics (−0.01)

Table 1. Cont.

Recommendations	Q-Value	Positive Contributions	Negative Contributions
job_DevOps_Architect_ [50,000_-_70,000_GBP] ,_Slough_	1.63	- programme work based on incoming orders (+0.02) - customs law (+0.02) - teach history (+0.02) - security threats (+0.02) - style sheet languages (+0.02)	- circular economy (−0.02) - direct inward dialing (−0.02) - address an audience (−0.01) - nanoelectronics (−0.01) - political science (−0.01)
job_Senior._NET_Software_ _Engineer	1.42	- joint ventures (+0.02) - perform procurement processes (+0.02) - energy efficiency (+0.02) - customer service (+0.02) - iOS (+0.02) - production processes (+0.02) - electrical machines (+0.01) - labour market (+0.01)	- seek innovation in current practices (−0.02) - history (−0.02)
job_Care_Assistant,_Tonbridge_	1.27	- represent the organisation (+0.03) - surveying (+0.03) - portfolio management in textile manufacturing (+0.02) - ecosystems (+0.02) - strategic planning (+0.02) - competition law (+0.02)	- perform maintenance procedures (−0.02) - LESS (−0.02) - metrology (−0.01) - make reservations (−0.01)
job_Localities_Social_Worker_-_Low_Caseload,_Berkshire_	1.25	- robotics (+0.02) - file documents (+0.02)	- physics (−0.03) - logistics (−0.03) - Slovak (−0.03) - security threats (−0.02) - Macedonian (−0.02) - Shiva (−0.02) - think creatively (−0.01) - adult education (−0.01)

Table 2. Recommendations and SHAP explanations.

Recommendations	Q-Value	Positive Contributions	Negative Contributions
job_System_Engineer_ _Athens	8.78	- alter management (+0.51) - literature (+0.35) - financial products (+0.31)	- soldering techniques (−0.38) - manage supplies (−0.34)
job_DevOps_Architect_ [50,000_-_70,000_GBP] ,_Slough_	1.63	- financial products (+0.10) - communicate with customers (+0.08)	- use questioning techniques (−0.09) - literature (−0.08) - Haskell (−0.08)
job_Senior._NET_Software_ _Engineer	1.42	- gather data (+0.12) - typography (+0.10) - communicate with customers (+0.09)	- use questioning techniques (−0.13) - Apache Tomcat (−0.09)
job_Care_Assistant,_Tonbridge_	1.27	- gather data (+0.13) - learning management systems (+0.10) - digital printing (+0.08) - customer insight (+0.06)	- use questioning techniques (−0.14)
job_Localities_Social_Worker_-_Low_Caseload,_Berkshire_	1.25	- digital printing (+0.08) - learning management systems (+0.07) - guarantee customer satisfaction (+0.06)	- use questioning techniques (−0.09) - optical character recognition software (−0.06)

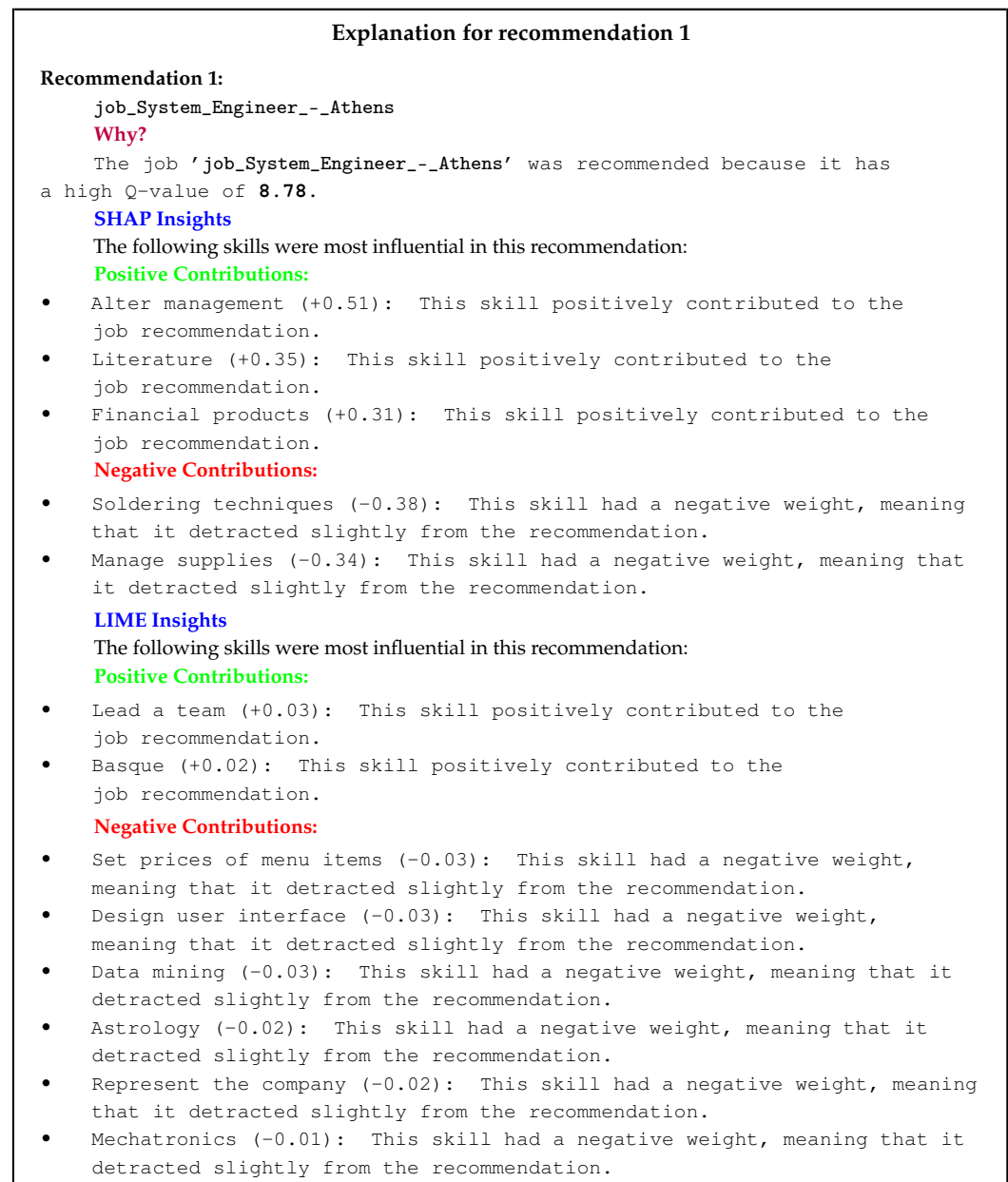


Figure 13. Explanation for job recommendation 1.

6. Conclusions

In conclusion, this research provides XR^2K^2G , a new framework for RS that integrates explainability, RS, and RL to improve the interpretability of recommendations. The component XR^2 describes the core methodologies: explainability (X) as an important element to understand recommendations, recommender systems (first R) as the foundational mechanism, and reinforcement learning (second R) for iterative improvement of recommendation quality. The K^2G architecture uses knowledge graphs (K), knowledge distillation (K) and graph-based methods (G), such as graph attention networks (GATs), to model complex relationships within the data effectively. Our analysis demonstrates that XR^2K^2G not only provides accurate job recommendations but also offers valuable insight through interpretable explanations.

This paper presents significant published studies related to our proposed approach and provides valuable information for our research. We present a brief overview of the XR^2K^2G framework, including RS, XAI, KG, RL, and KD. This study proposes XR^2K^2G , a

framework for job recommendations that provides clear and succinct explanations. This paper describes the materials and methods used to implement the proposed framework, including a description of the dataset and an experimental configuration of the system. We give our interpretation of the findings, explain what they mean, and outline the limitations of the study.

The XR^2K^2G approach has its drawbacks but gives a new integration of the explainability, RL, KD, and GAT methods along with a KG approach. A drawback is that, although the model can detect complex relationships among components, implicit feedback such as user click streams or session logs is still not considered, although these variables might further enhance personalization. Interpretability is another drawback. Although feature influence can be visualized with the aid of SHAP and LIME, explanation integration across several decision layers (e.g., embeddings, GAT, Q-learning) is not straightforward and could end up with end users receiving fragmented insights. Furthermore, although skill and occupation mappings resolve the trade-off between relevance and diversity of recommendations, additional work is needed, particularly when cold-starting new users or jobs. Finally, there is more to be said about ethical issues such as transparency in automated hiring, bias in skill requirements, and fairness. All of these will be necessary for broad applicability, as will measuring confidence in AI recommendations.

In future work, we plan to extend our framework by incorporating more personalized explanations derived from user-specific analyses. This enhancement aims to further improve the user experience and trust in the RS. In addition, we intend to develop a real-world application to validate the effectiveness of the framework in practical scenarios.

Author Contributions: Conceptualization, A.V.-A.; methodology, A.V.-A. and C.B.; software, A.V.-A.; validation, A.V.-A. and C.B.; formal analysis, A.V.-A. investigation, A.V.-A., I.M. and C.B.; resources, A.V.-A. and I.M.; data curation, A.V.-A. and I.M.; visualization, A.V.-A.; writing—original draft preparation, A.V.-A.; writing—review and editing, A.V.-A. and C.B. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Informed consent was obtained from all subjects involved in the study.

Data Availability Statement: The data presented in this study are available on request from the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Scherer, J.; Kiparski, G. Buchbesprechungen. Feiler, Lukas/Forgó, Nikolaus/Weigl, Michaela: The Eu General Data Protection Regulation (Gdpr): A Commentary. *Comput. Und Recht* **2018**, *34*, 69–70. [\[CrossRef\]](#)
2. McCrie, R.; Lee, S.Z. Decisions on Hiring to Meet Protective Goals. In *Security Operations Management*, 4th ed.; McCrie, R., Lee, S.Z., Eds.; Butterworth-Heinemann: Oxford, UK, 2022; pp. 71–118. [\[CrossRef\]](#)
3. Suadicani, P.; Bonde, J.P.; Olesen, K.; Gyntelberg, F. Job satisfaction and intention to quit the job. *Occup. Med.* **2013**, *63*, 96–102.
4. Vellido, A. The importance of interpretability and visualization in machine learning for applications in medicine and health care. *Neural Comput. Appl.* **2020**, *32*, 18069–18083. [\[CrossRef\]](#)
5. Brigo, D.; Huang, X.; Pallavicini, A.; de Ocariz Borde, H.S. Interpretability in deep learning for finance: A case study for the Heston model. *arXiv* **2021**, arXiv:2104.09476. [\[CrossRef\]](#)
6. Pessach, D.; Singer, G.; Avrahami, D.; Ben-Gal, H.C.; Shmueli, E.; Ben-Gal, I. Employees recruitment: A prescriptive analytics approach via machine learning and mathematical programming. *Decis. Support Syst.* **2020**, *134*, 113290. [\[CrossRef\]](#) [\[PubMed\]](#)
7. Linardatos, P.; Papastefanopoulos, V.; Kotsiantis, S. Explainable AI: A Review of Machine Learning Interpretability Methods. *Entropy* **2021**, *23*, 18. [\[CrossRef\]](#)

8. Raghavan, M.; Barocas, S.; Kleinberg, J.M.; Levy, K. Mitigating bias in algorithmic hiring: Evaluating claims and practices. In Proceedings of the FAT*’20: Conference on Fairness, Accountability, and Transparency, Barcelona, Spain, 27–30 January 2020; Hildebrandt, M., Castillo, C., Celis, L.E., Ruggieri, S., Taylor, L., Zanfir-Fortuna, G., Eds.; ACM: New York, NY, USA, 2020; pp. 469–481. [\[CrossRef\]](#)
9. Schemmer, M.; Kühl, N.; Satzger, G. Intelligent Decision Assistance Versus Automated Decision-Making: Enhancing Knowledge Workers Through Explainable Artificial Intelligence. In Proceedings of the 55th Hawaii International Conference on System Sciences, HICSS 2022, Virtual Event, Maui, HI, USA, 4–7 January 2022; ScholarSpace: Ann Arbor, MI, USA, 2022; pp. 1–10.
10. Ye, H.; Vedula, S.; Chen, Y.; Yang, Y.; Bronstein, A.M.; Dreslinski, R.G.; Mudge, T.N.; Talati, N. GRACE: A Scalable Graph-Based Approach to Accelerating Recommendation Model Inference. In Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2023, Vancouver, BC, Canada, 25–29 March 2023; Aamodt, T.M., Jerger, N.D.E., Swift, M.M., Eds.; ACM: New York, NY, USA, 2023; pp. 282–301. [\[CrossRef\]](#)
11. Kostis, I.; Sarafis, D.; Karamitsios, K.; Kotrotsios, K.; Kravari, K.; Badica, C.; Chatzimisios, P. Towards an Integrated Retrieval System to Semantically Match CVs, Job Descriptions and Curricula. In Proceedings of the 26th Pan-Hellenic Conference on Informatics, PCI 2022, Athens, Greece, 25–27 November 2022; ACM: New York, NY, USA, 2022; pp. 151–157. [\[CrossRef\]](#)
12. Li, Q.; Xia, W.; Yin, L.; Jin, J.; Yu, Y. Privileged Knowledge State Distillation for Reinforcement Learning-based Educational Path Recommendation. In Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2024, Barcelona, Spain, 25–29 August 2024; Baeza-Yates, R., Bonchi, F., Eds.; ACM: New York, NY, USA, 2024; pp. 1621–1630. [\[CrossRef\]](#)
13. Liu, H.; Sun, Z.; Qu, X.; Yuan, F. Top-aware recommender distillation with deep reinforcement learning. *Inf. Sci.* **2021**, *576*, 642–657. [\[CrossRef\]](#)
14. Adomavicius, G.; Tuzhilin, A. Toward the Next Generation of Recommender Systems: A Survey of the State-of-the-Art and Possible Extensions. *IEEE Trans. Knowl. Data Eng.* **2005**, *17*, 734–749. [\[CrossRef\]](#)
15. Ribeiro, M.T.; Singh, S.; Guestrin, C. “Why Should I Trust You?”: Explaining the Predictions of Any Classifier. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, New York, NY, USA, 13–17 August 2016; KDD’16, pp. 1135–1144. [\[CrossRef\]](#)
16. Lundberg, S.M.; Lee, S. A Unified Approach to Interpreting Model Predictions. In Proceedings of the Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, Long Beach, CA, USA, 4–9 December 2017; Guyon, I., von Luxburg, U., Bengio, S., Wallach, H.M., Fergus, R., Vishwanathan, S.V.N., Garnett, R., Eds.; ACM: New York, NY, USA, 2017; pp. 4765–4774.
17. Vultureanu-Albisi, A.; Badica, C. Improving Students’ Performance by Interpretable Explanations using Ensemble Tree-Based Approaches. In Proceedings of the 15th IEEE International Symposium on Applied Computational Intelligence and Informatics, SACI 2021, Timisoara, Romania, 19–21 May 2021; IEEE: Piscataway, NJ, USA, 2021; pp. 215–220. [\[CrossRef\]](#)
18. Bouleanu, D.; Badica, C.; Kravari, K. Using SHAP-Based Interpretability to Understand Risk of Job Changing. In Proceedings of the Intelligent Distributed Computing XV, 15th International Symposium on Intelligent Distributed Computing, IDC 2022, Virtual Event, Bremen, Germany, 14–15 September 2022; Braubach, L., Jander, K., Badica, C., Eds.; Studies in Computational Intelligence; Springer: Berlin/Heidelberg, Germany, 2022; Volume 1089, pp. 41–50. [\[CrossRef\]](#)
19. Wang, X.; Wang, D.; Xu, C.; He, X.; Cao, Y.; Chua, T. Explainable Reasoning over Knowledge Graphs for Recommendation. In Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019, The Thirty-First Innovative Applications of Artificial Intelligence Conference, IAAI 2019, The Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2019, Honolulu, HI, USA, 27 January–1 February 2019; AAAI Press: Washington, DC, USA, 2019; pp. 5329–5336. [\[CrossRef\]](#)
20. Xian, Y.; Fu, Z.; Muthukrishnan, S.; de Melo, G.; Zhang, Y. Reinforcement Knowledge Graph Reasoning for Explainable Recommendation. In Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2019, Paris, France, 21–25 July 2019; Piwowarski, B., Chevalier, M., Gaussier, É., Maarek, Y., Nie, J., Scholer, F., Eds.; ACM: New York, NY, USA, 2019; pp. 285–294. [\[CrossRef\]](#)
21. Wang, P.; Fan, Y.; Xia, L.; Zhao, W.X.; Niu, S.; Huang, J.X. KERL: A Knowledge-Guided Reinforcement Learning Model for Sequential Recommendation. In Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2020, Virtual Event; Xi’an, China, 25–30 July 2020; ACM: New York, NY, USA, 2020; pp. 209–218. [\[CrossRef\]](#)
22. Kang, S.; Hwang, J.; Kweon, W.; Yu, H. DE-RRD: A Knowledge Distillation Framework for Recommender System. In Proceedings of the CIKM’20: The 29th ACM International Conference on Information and Knowledge Management, Virtual Event, Ireland, 19–23 October 2020; d’Aquin, M., Dietze, S., Hauff, C., Curry, E., Cudré-Mauroux, P., Eds.; ACM: New York, NY, USA, 2020; pp. 605–614. [\[CrossRef\]](#)
23. Hogan, A.; Blomqvist, E.; Cochez, M.; d’Amato, C.; de Melo, G.; Gutiérrez, C.; Kirrane, S.; Labra Gayo, J.E.; Navigli, R.; Neumaier, S.; et al. *Knowledge Graphs*; Number 22 in Synthesis Lectures on Data, Semantics, and Knowledge; Springer: Berlin/Heidelberg, Germany, 2021. [\[CrossRef\]](#)

24. Wang, Q.; Mao, Z.; Wang, B.; Guo, L. Knowledge Graph Embedding: A Survey of Approaches and Applications. *IEEE Trans. Knowl. Data Eng.* **2017**, *29*, 2724–2743. [\[CrossRef\]](#)
25. Asmara, S.M.; Sahabudin, N.A.; Ismail, N.S.N.B.; Sabri, I.A.A. A Review of Knowledge Graph Embedding Methods of TransE, TransH and TransR for Missing Links. In Proceedings of the 8th IEEE International Conference On Software Engineering and Computer Systems, ICSECS 2023, Penang, Malaysia, 25–27 August 2023; IEEE: Piscataway, NJ, USA, 2023; pp. 470–475. [\[CrossRef\]](#)
26. Velickovic, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; Bengio, Y. Graph Attention Networks. *arXiv* **2017**, arXiv:1710.10903.
27. Knyazev, B.; Taylor, G.W.; Amer, M.R. Understanding attention in graph neural networks. *arXiv* **2019**, arXiv:1905.02850.
28. Shi, M.; Qin, F.; Ye, Q.; Han, Z.; Jiao, J. A scalable convolutional neural network for task-specified scenarios via knowledge distillation. *arXiv* **2016**, arXiv:1609.05695.
29. Levine, S.; Kumar, A.; Tucker, G.; Fu, J. Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems. *arXiv* **2020**, arXiv:2005.01643.
30. Fan, J.; Wang, Z.; Xie, Y.; Yang, Z. A Theoretical Analysis of Deep Q-Learning. In Proceedings of the 2nd Annual Conference on Learning for Dynamics and Control, L4DC 2020, Online Event, Berkeley, CA, USA, 11–12 June 2020; Bayen, A.M., Jadbabaie, A., Pappas, G.J., Parrilo, P.A., Recht, B., Tomlin, C.J., Zeilinger, M.N., Eds.; Proceedings of Machine Learning Research: Brooklyn, NY, USA, 2020; Volume 120, pp. 486–489.
31. Muraretu, I.; Bouleanu, D.; Vultureanu-Albisi, A.; Badica, C.; Sarafis, D.; Kravari, K.; Chatzimisios, P. A Microservice-Based Multi-agent System for the Job Market. In Proceedings of the Intelligent Distributed Computing XVI-16th International Symposium on Intelligent Distributed Computing, IDC 2023, Hamburg, Germany, 13–15 September 2023; Köhler-Bußmeier, M., Renz, W., Sudeikat, J., Eds.; Springer: Berlin/Heidelberg, Germany, 2023; Volume 1138, pp. 301–316. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.